

When Batching Backfires: The Hidden Cost of Adding a Gateway to Continuous LLM Serving

Wasiq Nauman Qureshi
Independent Researcher

Abstract

Production LLM stacks often add request batching at an API gateway even though engines such as vLLM already continuously batch sequences at token granularity. This creates two schedulers: the gateway can report larger batches while delaying requests that the engine could have admitted immediately. We ask when this layered design produces useful coalescing and when it merely moves queueing in front of the GPU. We present ADIP, a compact gateway and measurement artifact for isolating this interaction on a single consumer-grade GPU. Our central hypothesis is that, below saturation, intentional micro-waiting is too short relative to inter-arrival time to create useful batches; multi-request gateway batches instead emerge from backlog accumulated while a serial gateway worker awaits the backend. We test this claim with a predeclared, open-loop matrix comparing direct vLLM, gateway pass-through, a fixed 1 ms policy, and a frozen adaptive policy over four offered loads and three run-level repetitions. The study records request traces, gateway queue and backend time, batch formation, available GPU telemetry, and harness validity. Across 6,912 measured requests, all 48 conditions completed without a request failure or harness exclusion. Above 0.5 requests/s, fixed outer batching formed 1.54–2.90 prompts per backend call but increased paired p95 latency by 1.05–1.51 seconds relative to pass-through; the added gateway queue averaged 0.77–0.83 seconds. A serial-occupancy model predicts group size with 0.86% mean absolute percentage error across 24 batched runs, while the adaptive policy provides no resolved p95 improvement over fixed batching. The result is a reproducible account of why a larger gateway batch is not necessarily evidence of better serving: it can be a symptom of withheld scheduling opportunity in front of an engine that already batches continuously.

1 Introduction

An LLM request is not a single GPU operation. It begins with a parallel prefill over the prompt and continues as an autoregressive sequence of decode iterations. Serving engines exploit this structure by admitting and retiring requests between iterations. Orca introduced iteration-level scheduling [7]; vLLM couples continuous scheduling with PagedAttention to reduce key–value-cache waste [4]. These engines therefore already decide which requests share the GPU at fine granularity.

The deployment stack often adds a second scheduler. An HTTP gateway may hold compatible requests for a few milliseconds, concatenate their prompts into one backend call, and then split the outputs. This pattern is attractive because batching can amortize HTTP, tokenization, and launch overhead, and it has a long history in general prediction serving [2]. But in front of a continuously batching LLM engine, the gateway does not create GPU batching from nothing. It changes the time and shape in which work becomes visible to an engine that already batches. We call this interaction *double batching*.

Double batching creates a non-obvious trade-off. Waiting can expose several prompts in one HTTP request, but it also withholds those prompts from the backend scheduler. A longer gateway batch can reduce per-prompt backend overhead while worsening time to first token and head-of-line blocking. The gateway sees only request-level completion, whereas the engine sees prefill, decode, KV-cache pressure, and token-level progress. Optimizing the outer scheduler without measuring the inner scheduler can therefore optimize the wrong bottleneck.

This paper tests one falsifiable systems hypothesis:

H: At offered loads that direct vLLM can fully admit, a millisecond gateway window is too short relative to inter-arrival time to form useful batches. Multi-request outer batches, when they appear, are primarily backlog coalescing caused by the gateway awaiting a backend call. Consequently, outer batching improves latency only if the backend-time reduction from coalescing exceeds the queueing delay and loss of scheduling freedom imposed on vLLM.

The null hypothesis is operationally useful: fixed and adaptive gateway batching provide no repeatable tail-latency advantage over pass-through at matched offered load. A positive result requires a paired, run-level improvement visible across repetitions and a latency decomposition that repays added queueing; a negative result is evidence that the engine, not the gateway, should own scheduling for the tested regime.

We study the question with ADIP, an asynchronous gateway and measurement artifact. The final matrix compares four paths—direct vLLM, gateway pass-through, fixed-window batching, and a frozen adaptive policy—at 0.5, 1.0, 1.5, and 1.8 requests/s. Every condition uses identical prompts, generation parameters, open-loop arrivals, and a single consumer-grade GPU. Three seeds serve as the replication unit. We retain request-level latency, queue and backend decomposition, batch identifiers and sizes, available GPU telemetry, failures, and arrival drift.

The paper makes four contributions:

1. It identifies a precise gap between engine-level continuous batching and request-level gateway batching, and gives a simple model separating *window formation* from *backlog formation*.
2. It provides a tested implementation of fixed and adaptive outer scheduling with compatibility keys, deadline-aware closure, structured telemetry, and explicit cancellation and shutdown semantics.
3. It evaluates the schedulers using predeclared open-loop arrivals and run-level uncertainty rather than treating requests from one run as independent replications.
4. It records the complete validity trail, including interrupted and harness-rejected attempts, and binds final claims to machine-readable analysis artifacts so neutral or negative findings remain auditable.

The scope is intentionally narrow. We do not propose a replacement for vLLM, claim production-scale generality, or tune on the final matrix. The goal is a defensible measurement study that explains a common layered-serving decision on hardware available to an individual researcher.

2 Background and Related Work

2.1 Why LLM batching is different

Conventional request-level batching waits for several inputs, executes one tensor program, and releases the batch when all outputs are ready. Generative inference has two asymmetric phases. Prefill processes many prompt tokens in parallel and is typically compute intensive; decode processes one new token per active sequence and is often memory-bandwidth limited. Request lengths vary, so static batches suffer from head-of-line blocking. Orca’s iteration-level scheduler allows the active set to change between decode iterations [7]. PagedAttention reduces fragmentation in the dynamically growing KV cache and enables vLLM to sustain larger active sets [4].

Recent systems optimize this inner schedule. Sarathi-Serve uses chunked prefills and stall-free schedules to reduce prefill–decode interference [1]. DeepSpeed-FastGen’s Dynamic SplitFuse also decomposes prompts and generations to balance throughput and latency [3]. DistServe separates prefill and decode resources to optimize goodput under distinct latency objectives [8]. These systems have detailed visibility into token-level execution. ADIP does not: it treats vLLM as a black-box HTTP backend and asks what an outer layer can accomplish with request-level signals alone.

2.2 Layered prediction serving

Clipper demonstrated that a serving layer can batch calls to otherwise unmodified ML frameworks and adapt batch size to a latency objective [2]. That result motivates gateway batching, but the backend models in classical prediction serving usually execute a bounded forward pass. An LLM backend is itself a stateful scheduler. Outer batching therefore changes both transport granularity and the arrival process observed by the inner scheduler.

The workload also matters. BurstGPT shows that real LLM services exhibit time-varying concurrency, response lengths, and failures, and warns that scheduling conclusions may not generalize across traces [6]. Our primary experiment deliberately uses a deterministic constant-rate workload. This sacrifices trace realism to isolate the mechanism; burst sensitivity is left as future validation rather than being implied by the constant-rate result.

2.3 Window formation versus backlog formation

Let λ be the request arrival rate and w the gateway’s intentional wait window. The primary workload schedules requests periodically, so its nominal inter-arrival gap is

$$\Delta = \lambda^{-1}. \quad (1)$$

In the absence of backlog or harness jitter, the number of companions that can arrive during a window is exactly zero whenever $w < \Delta$. At the largest tested rate, 1.8 requests/s, $\Delta = 555.6$ ms: both the 1 ms fixed window and the 20 ms adaptive cap close hundreds of milliseconds before the next scheduled arrival. Thus, the primary workload makes a sharper prediction than “small batches are unlikely”: intentional waiting alone should form no multi-request batch.

For comparison, if arrivals were memoryless rather than periodic, a Poisson approximation gives

$$\mathbb{E}[N_w] = \lambda w, \quad P(N_w \geq 1) = 1 - e^{-\lambda w}, \quad (2)$$

where N_w is the number of companion arrivals during the window. At 1.8 requests/s, a 1 ms window yields $\mathbb{E}[N_w] = 0.0018$ and $P(N_w \geq 1) \approx 0.18\%$. Even a 20 ms cap yields only 0.036 expected companions and a 3.54% probability. The Poisson case is therefore a conservative stress intuition, not the data-generating model used for the confirmatory matrix.

Now let S be the time for which a serial gateway worker awaits one backend call. Requests continue to arrive during that interval, creating on the order of λS queued companions (up to the phase and integer boundary of the periodic schedule). When the worker returns and drains the ingress queue, it can dispatch a multi-request backend call even if w is nearly zero. This *backlog coalescing* is qualitatively different from Eq. 2: its control variable is service occupancy, not the micro-wait. It also exposes the cost of a serial outer scheduler. A request that arrives behind an in-flight call cannot reach vLLM until the outer worker returns, even though vLLM could otherwise admit it into continuous batching.

This gives a quantitative prediction in the continuously busy regime. Let $\bar{S}_{\text{call}}$ be mean backend occupancy weighted once per outer call. The next call’s expected group size should track

$$\hat{B}_{\text{occupancy}} = \max(1, \lambda \bar{S}_{\text{call}}), \quad (3)$$

where the floor at one represents the request that begins a cycle after an idle period. Agreement with Eq. 3, despite $w \ll \Delta$, is evidence for backlog coalescing; systematic excess over it would suggest intentional waiting or residual queue growth.

Our evaluation tests which mechanism dominates by jointly examining observed batch size, gateway queue time, backend time, completion of offered work, and tail latency. This mechanism-level accounting is the paper’s main distinction from a benchmark that merely ranks configurations.

3 Research Questions and Methodology

3.1 Research questions

We organize the evaluation around three questions.

RQ1: Layering cost.

What latency change is introduced by a pass-through gateway relative to direct vLLM?

RQ2: Outer batching.

At matched offered load, does fixed gateway batching improve end-to-end latency or backend time after accounting for queueing and the backend’s own continuous batching?

RQ3: Adaptation.

Does a frozen low-load bypass policy avoid harmful waiting while preserving useful backlog coalescing?

For RQ2 and RQ3, the confirmatory comparison is paired by repetition seed. The null is zero mean paired difference. We report effect direction and 95% Student- t intervals over three run-level differences; we do not label a difference statistically significant from request counts within a run.

Table 1: Frozen primary experiment. Every cell has three run-level repetitions.

Factor	Value
GPU	NVIDIA RTX 3060, 12 GB
Model	Qwen2.5-1.5B-Instruct, float16
Backend	vLLM 0.29.0, eager execution
Modes	direct, pass-through, fixed, adaptive
Offered rates	0.5, 1.0, 1.5, 1.8 requests/s
Prompt bucket	8–128 tokenizer-derived input tokens
Output cap	64 tokens
Warmup	50 sequential requests per condition
Measurement	120 seconds, open loop
Repetitions	seeds 1729, 2718, 3141
Fixed policy	max batch 8, max wait 1 ms
Adaptive policy	max batch 8, max wait 20 ms

3.2 Testbed

All measurements run locally in WSL2 Ubuntu on a Windows host with an NVIDIA GeForce RTX 3060 12 GB. The backend is vLLM 0.29.0 in eager mode with float16 weights, maximum model length 2,048, GPU-memory utilization 0.85, and at most eight sequences. The model is **Qwen2.5-1.5B-Instruct** at immutable revision 989aa7980e4cf806f80c7fef2b1adb7bc71aa306 [5]. Generation is deterministic ($T = 0$) with at most 64 output tokens. Direct traffic targets port 8001; gateway modes target ADIP on port 8000, which forwards to the same backend.

3.3 Workload and controls

The workload is synthetic and length controlled, not a production trace. Four project-authored seed passages are expanded deterministically until they fall inside the 8–128 token bucket under the exact model tokenizer. All requests use the same model, output cap, and temperature, making them compatible for gateway batching. The benchmark generates the complete constant-rate arrival schedule before execution. Each request sleeps until an absolute target time; later requests do not wait for earlier completions. This open-loop design avoids coordinated omission when the system slows down.

Fifty warmup requests precede each measured interval. Warmups are excluded from every aggregate. A five-second cooldown separates conditions. The final matrix contains $4 \times 4 \times 3 = 48$ conditions and 5,760 seconds of measured traffic, excluding warmup and cooldown. The fixed 1 ms comparator and adaptive parameters were frozen from calibration and pilot data before the final matrix. The runner executes the declared order by mode (direct, pass-through, fixed, adaptive), then repetition, then increasing rate. This deterministic order makes interruption recovery auditable but does not balance slow temporal or thermal drift. Seed pairing controls workload construction, not wall-clock position; we treat the lack of randomized interleaving as a validity limitation rather than implying otherwise.

3.4 Validity gate and failure accounting

Host scheduling can invalidate an open-loop experiment even when the server returns success. We therefore record target and actual arrival time for every request. A condition is marked `invalid_harness` when more than 1% of measured requests arrive over 50 ms late. The raw records remain stored, but the condition is excluded from performance claims. Ordinary incomplete directories are rejected by the resume logic rather than appended to. Request failures remain in the denominator and are never converted to zero latency.

3.5 Metrics and uncertainty

The request is the latency observation unit; the run is the replication unit. For each run we compute successful request count divided by the 120-second schedule, output tokens/s, success and failure counts, p50/p95/p99 end-to-end latency, mean and p95 gateway queue time, mean backend time, mean batch size, GPU utilization, peak VRAM, power, and p95 arrival drift. The first quantity is a count-normalized load-completion check: because the harness waits for every scheduled request, it equals offered load when all requests succeed. It is not an independent estimate

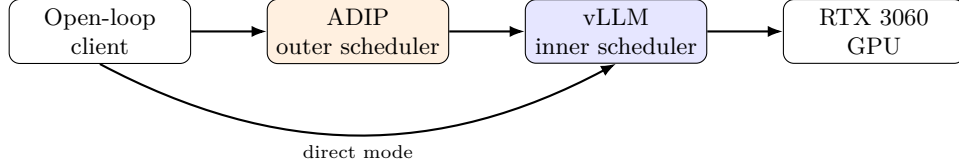


Figure 1: The experimental boundary. Gateway batching controls when and in what HTTP grouping prompts become visible; vLLM independently controls token-level GPU scheduling.

of completion capacity or saturation throughput. We then average the three run-level values for each condition and report a two-sided 95% Student- t interval. With only three repetitions, these intervals are necessarily wide; figures show each run point to prevent the mean from hiding instability.

Each request record carries its outer batch ID and reported batch size. A request-wise average of that field is size biased: a group of four contributes four observations while a singleton contributes one. For mechanism claims we therefore group records by batch ID and report the call-weighted mean number of prompts per backend HTTP call. The raw request-weighted mean remains in each run summary for auditability but is not labeled as a call-level average. The same unique-call grouping yields $\bar{S}_{\text{call}}$ for Eq. 3; the analyzer reports measured group size, predicted group size, and their ratio for every fixed and adaptive run.

The primary latency effects are paired by rate and seed:

$$\Delta_{\text{proxy}} = M_{\text{pass}} - M_{\text{direct}}, \quad (4)$$

$$\Delta_{\text{fixed}} = M_{\text{fixed}} - M_{\text{pass}}, \quad (5)$$

$$\Delta_{\text{adaptive}} = M_{\text{adaptive}} - M_{\text{fixed}}, \quad (6)$$

for metric M . Absolute and relative deltas are both retained. We interpret the count-normalized completion check separately from goodput, service capacity, and saturation throughput.

4 ADIP Design

4.1 Two scheduling layers

Figure 1 shows the paths under comparison. Direct mode sends one OpenAI-compatible completion per client request. Pass-through adds HTTP translation and telemetry but calls the backend once per request. Fixed and adaptive modes place requests in compatibility-keyed queues and send one backend prompt list per outer batch. In every mode, vLLM retains control of its token-level continuous scheduler.

4.2 Compatibility and queue discipline

A batch key consists of model identifier, output-token cap, and temperature. Only equal keys share a backend request. The ingress queue is bounded; a full queue rejects before creating unresolved work. One coordinator drains ingress into per-key FIFO queues, selects the oldest compatible queue, obtains a pure policy decision, and awaits one backend batch. This simple coordinator makes the mechanism measurable, but it is intentionally serial: arrivals during a backend call accumulate until that call returns. Section 2.3 predicts that this occupancy-driven backlog can dominate the nominal micro-wait.

4.3 Fixed batch closure

For the oldest compatible queue, the fixed policy dispatches when (i) the batch reaches eight requests, (ii) the oldest request reaches the 1 ms window, or (iii) further waiting would consume estimated service slack before the earliest soft deadline. Otherwise it waits only for the remaining window. Policy code does not perform I/O or sleep; the coordinator owns timing and queue mutation.

4.4 Adaptive closure

The adaptive policy was frozen before final evaluation. It maintains exponentially weighted moving averages of compatible arrival rate $\hat{\lambda}$ and backend time $\hat{S}$ with smoothing factor 0.2. For remaining window w_r , it computes

expected companions $\hat{\lambda}w_r$. If this value is below 1.0, it dispatches immediately. Otherwise it waits for the minimum of the remaining 20 ms cap, estimated time to fill the batch,

$$t_{\text{fill}} = \frac{B_{\text{max}} - B_{\text{queue}}}{\hat{\lambda}}, \quad (7)$$

and positive deadline slack after reserving $\hat{S}$. Full queues, expired windows, and exhausted slack dispatch immediately. Importantly, low-load bypass does not split requests already accumulated during an in-flight backend call; it closes the existing backlog without adding intentional wait.

4.5 Instrumentation and failure semantics

Every accepted request receives a UUID and optional experiment identifier. Structured events record enqueue, policy reason, batch dispatch, backend completion, and request completion without prompt text or credentials. The response carries batch ID, batch size, queue time, backend time, total gateway time, and deadline outcome. Backend failures resolve every future in the affected batch. Cancellation and shutdown settle queued and in-flight futures exactly once, preventing a successful test run from hiding leaked asynchronous work.

The benchmark writes request JSONL as each request completes, while a separate sampler records GPU utilization, VRAM, power, and temperature through `nvidia-smi`. A manifest created before traffic binds the run to the Git commit, model revision, mode, rate, seed, and frozen parameters. Atomic summary writes occur only after traffic and validity checks complete.

5 Evaluation

The analyzer accepted all 48 declared conditions and excluded none. Across 6,912 measured requests, every request completed successfully. No condition crossed the harness-invalid threshold; the largest condition-level p95 arrival drift was 2.280 ms. We report each scheduled condition, including neutral effects, rather than selecting rates after observing the results.

5.1 RQ1: What does the pass-through layer cost?

Pass-through minus direct paired p95 effects ranged from -451 to $+251$ ms, and every 95% interval crossed zero. This experiment therefore does not resolve a latency penalty from translation and telemetry alone. The paired p95 deltas (pass-through minus direct) were -180 ms [95% CI: $-676, 317$] at 0.5 requests/s, -451 ms [$-1,662, 761$] at 1.0 requests/s, $+162$ ms [$-309, 634$] at 1.5 requests/s, and $+251$ ms [$-424, 925$] at 1.8 requests/s. The direction changes with load, and the intervals are wide relative to the means. These data support using pass-through as the outer-scheduler baseline, but not the stronger claim that its overhead is zero.

5.2 RQ2: Does fixed outer batching help?

At 1.0, 1.5, and 1.8 requests/s, fixed batching increased paired p95 latency over pass-through by 1,510, 1,047, and 1,370 ms, respectively; all three 95% intervals exclude zero. The policy formed larger outer calls but did not deliver a consistent backend-time reduction, so 0.77–0.83 seconds of gateway queueing remained exposed to the request. At 0.5 requests/s, the fixed policy remained at 1.00 prompt per backend call and changed paired p95 latency by -123 ms [95% CI: $-754, 508$]. This is the predicted low-load regime: the 2-second periodic gap leaves no companion for a 1 ms window, no backlog develops, and the latency effect is unresolved.

The regime changes at 1.0 requests/s. Mean fixed-policy outer group size rises to 1.54, then to 2.24 and 2.90 at 1.5 and 1.8 requests/s. Yet mean p95 latency rises from 1,884 to 3,393 ms at 1.0 requests/s, from 2,172 to 3,220 ms at 1.5 requests/s, and from 2,120 to 3,490 ms at 1.8 requests/s, relative to pass-through. The paired increases are respectively 1,510 ms [1,330, 1,690], 1,047 ms [303, 1,791], and 1,370 ms [484, 2,256]. The means of the run-wise paired relative increases are 82.6%, 48.7%, and 65.1%, respectively.

Larger outer calls did not produce a consistent backend-time reduction. At 1.0, 1.5, and 1.8 requests/s, mean backend time changed from 1,502, 1,688, and 1,661 ms in pass-through to 1,578, 1,529, and 1,658 ms under fixed batching. Against those changes, fixed batching added mean gateway queue delays of 767, 788, and 833 ms. Thus the observed tail penalty is not a mysterious cost of HTTP aggregation: a serial gateway withholds arrivals while its current backend call runs, and the modest or absent backend saving does not repay that queue.

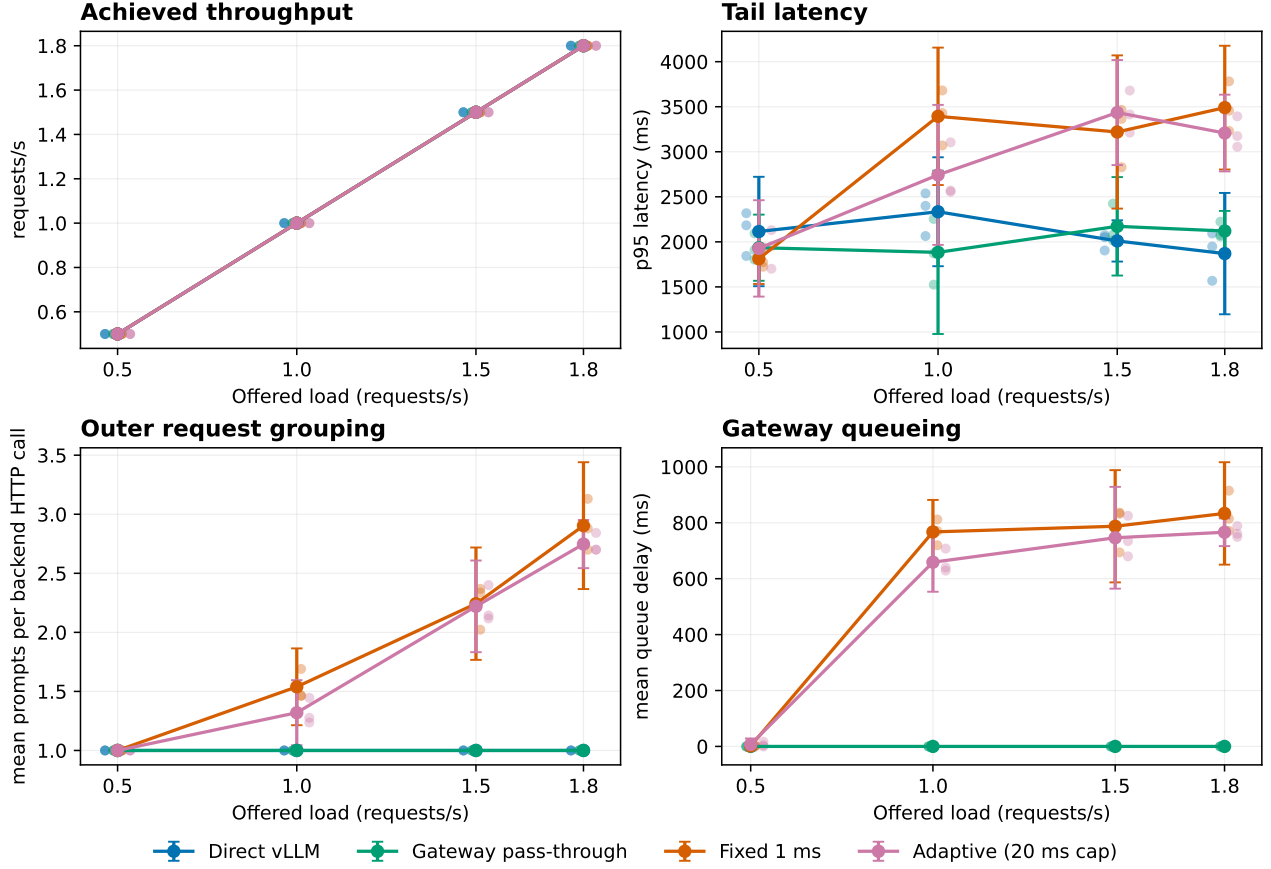


Figure 2: Primary matrix on `Qwen2.5-1.5B-Instruct` and a single consumer-grade GPU. Lines show means across three run-level repetitions; error bars are two-sided 95% Student- t intervals; translucent points are individual repetitions. “Throughput” is successful request count divided by the fixed 120-second schedule, not an independent saturation measurement. Outer grouping is call-weighted prompts per backend HTTP call. Direct mode has no gateway queue measurement.

The batch-formation data close the mechanism accounting. Across all 24 fixed and adaptive runs, Eq. 3 predicts call-weighted outer group size with 0.020 requests/call mean absolute error, 0.86% mean absolute percentage error, and descriptive $R^2 = 0.9985$. Individual measured-to-predicted ratios range from 0.978 to 1.000. Because $\bar{S}_{\text{call}}$ is measured in the same run, this agreement is a consistency diagnostic rather than an independent causal estimate. It is nevertheless discriminating here: intentional waits are at most 20 ms while scheduled gaps are at least 555.6 ms, whereas backend occupancy is long enough to account quantitatively for the observed groups.

All 6,912 requests completed, so the count-normalized completion metric equals offered load in every condition. This rules out request failure as the source of the latency differences, but it does not demonstrate equal saturation capacity: the metric uses the configured schedule duration rather than the wall-clock span from first arrival to last completion.

5.3 RQ3: Does the frozen adaptive policy help?

Adaptive-minus-fixed paired p95 effects ranged from -650 to $+216$ ms, with every 95% interval crossing zero. The frozen adaptive closure policy did not produce a repeatable tail-latency advantage. The paired adaptive-minus-fixed p95 deltas were $+115$ ms [95% CI: $-696, 926$], -650 ms [$-1,669, 369$], $+216$ ms [$-1,157, 1,589$], and -282 ms [$-995, 431$] as load increased. Adaptive closure reduced mean outer grouping from 1.54 to 1.32 at 1.0 requests/s and from 2.90 to 2.75 at 1.8 requests/s, but it left grouping nearly unchanged at 1.5 requests/s and none of the paired latency intervals excluded zero. With three repetitions, the defensible result is not that adaptive and fixed are equivalent; it is that this frozen policy did not demonstrate a repeatable advantage.

Table 2: All primary conditions. Values are run-level mean $\pm$ 95% Student- t half-width over three repetitions. The throughput column is the successful request count divided by the 120-second arrival schedule.

Mode	Rate (req/s)	Throughput (req/s)	p95 latency (ms)	Outer group (requests)	Queue delay (ms)
Direct vLLM	0.5	0.50 ± 0.00	2115 ± 607	1.00 ± 0.00	—
Direct vLLM	1.0	1.00 ± 0.00	2334 ± 605	1.00 ± 0.00	—
Direct vLLM	1.5	1.50 ± 0.00	2010 ± 229	1.00 ± 0.00	—
Direct vLLM	1.8	1.80 ± 0.00	1870 ± 674	1.00 ± 0.00	—
Gateway pass-through	0.5	0.50 ± 0.00	1935 ± 367	1.00 ± 0.00	0 ± 0
Gateway pass-through	1.0	1.00 ± 0.00	1884 ± 906	1.00 ± 0.00	0 ± 0
Gateway pass-through	1.5	1.50 ± 0.00	2172 ± 546	1.00 ± 0.00	0 ± 0
Gateway pass-through	1.8	1.80 ± 0.00	2120 ± 223	1.00 ± 0.00	0 ± 0
Fixed 1 ms	0.5	0.50 ± 0.00	1812 ± 280	1.00 ± 0.00	2 ± 1
Fixed 1 ms	1.0	1.00 ± 0.00	3393 ± 763	1.54 ± 0.33	767 ± 114
Fixed 1 ms	1.5	1.50 ± 0.00	3220 ± 850	2.24 ± 0.48	788 ± 201
Fixed 1 ms	1.8	1.80 ± 0.00	3490 ± 687	2.90 ± 0.54	833 ± 183
Adaptive (20 ms cap)	0.5	0.50 ± 0.00	1927 ± 535	1.00 ± 0.00	7 ± 21
Adaptive (20 ms cap)	1.0	1.00 ± 0.00	2744 ± 777	1.32 ± 0.28	659 ± 106
Adaptive (20 ms cap)	1.5	1.50 ± 0.00	3435 ± 583	2.22 ± 0.39	746 ± 182
Adaptive (20 ms cap)	1.8	1.80 ± 0.00	3208 ± 426	2.75 ± 0.20	766 ± 50

5.4 Validity and negative-result accounting

The terminal audit found 48 complete, zero harness-invalid, and zero failed conditions. Interrupted directories were archived before resume and do not enter any aggregate. Every run records the same immutable model revision and a clean source tree. The first two direct runs precede a harness commit that added safe resume and explicit invalid-state bookkeeping; a source diff confirmed that request generation, warmup, open-loop timing, clients, and summarization were unchanged.

GPU sampling is not complete enough for a matrix-level result. Of 5,726 GPU monitor records, 738 contain `FileNotFoundError` after the restart, leaving fewer than three telemetry-bearing adaptive repetitions at some rates. The analysis JSON retains available values and their actual n , but the paper makes no comparative utilization, VRAM, power, or thermal claim. Request-level latency, queue, backend, batch, failure, and drift records are complete for all 48 conditions.

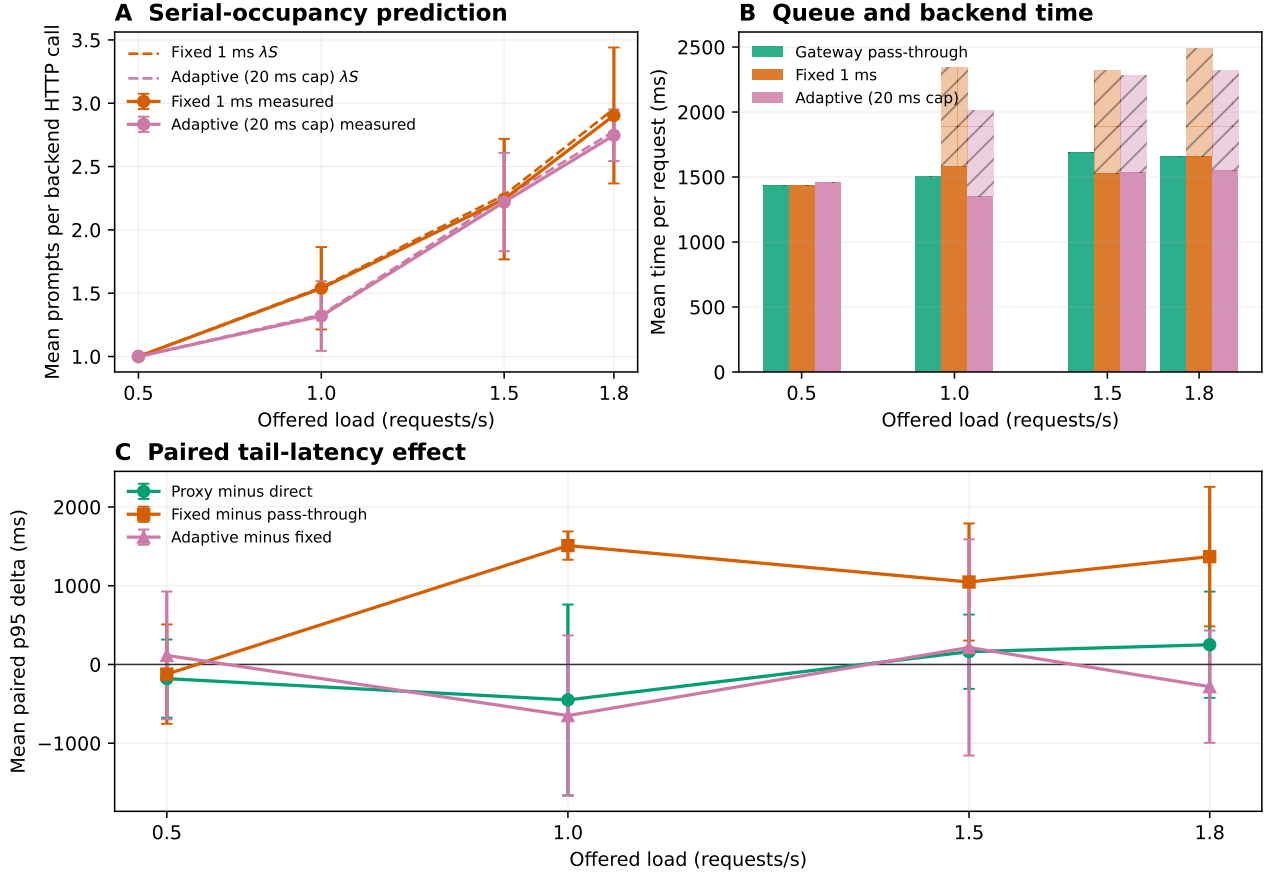


Figure 3: Mechanism accounting for Qwen2.5-1.5B-Instruct on a single consumer-grade GPU. Panel A compares measured call-weighted outer grouping with the serial-occupancy prediction $\max(1, \lambda \bar{S}_{\text{call}})$. Panel B stacks gateway queue time (hatched) above backend time. Panel C shows paired p95-latency effects; positive values mean the treatment is slower. Error bars are two-sided 95% Student- t intervals over three runs or paired differences.

Table 3: Paired p95-latency effects. Positive values mean the treatment is slower than its comparator. Values are mean $\pm$ 95% Student- t half-width across shared repetition seeds.

Comparison	Rate	p95 delta (ms)	Relative delta (%)
Proxy minus direct	0.5	-180 $\pm$ 496	-8.0 $\pm$ 20.9
Proxy minus direct	1.0	-451 $\pm$ 1212	-18.4 $\pm$ 46.4
Proxy minus direct	1.5	162 $\pm$ 472	8.0 $\pm$ 22.9
Proxy minus direct	1.8	251 $\pm$ 675	15.0 $\pm$ 42.3
Fixed minus pass-through	0.5	-123 $\pm$ 631	-5.7 $\pm$ 31.6
Fixed minus pass-through	1.0	1510 $\pm$ 180	82.6 $\pm$ 47.3
Fixed minus pass-through	1.5	1047 $\pm$ 744	48.7 $\pm$ 40.2
Fixed minus pass-through	1.8	1370 $\pm$ 886	65.1 $\pm$ 47.6
Adaptive minus fixed	0.5	115 $\pm$ 811	7.1 $\pm$ 45.0
Adaptive minus fixed	1.0	-650 $\pm$ 1019	-18.8 $\pm$ 26.1
Adaptive minus fixed	1.5	216 $\pm$ 1373	8.0 $\pm$ 47.6
Adaptive minus fixed	1.8	-282 $\pm$ 713	-7.8 $\pm$ 18.4

6 Discussion and Limitations

6.1 Interpreting a second scheduler

The crucial distinction is between forming a batch by waiting and forming one because the outer worker was already busy. The former can be tuned with a micro-window; the latter is a consequence of serial service occupancy. If observed batch size rises while intentional companion probability remains near zero, backlog coalescing is the parsimonious explanation. Whether that is beneficial depends on the full latency decomposition. Lower backend time does not imply lower end-to-end latency when the same policy increases queue time or withholds prompts from continuous batching.

The final measurements exhibit exactly this separation. At 1.0–1.8 requests/s, fixed batching raises call-weighted grouping from 1.54 to 2.90, but the occupancy prediction explains the grouping with 0.86% mean absolute percentage error. Backend time does not fall consistently, while mean gateway queue delay rises to 0.77–0.83 seconds. The resulting p95 penalties remain positive under paired run-level intervals. The failure is therefore not that the gateway cannot aggregate prompts; it aggregates them for the wrong reason and at the wrong scheduling boundary.

This distinction also changes how an adaptive policy should be judged. A low-load bypass can correctly decide never to wait and still emit multi-request batches after a backend call. Calling such behavior “adaptive waiting” would be misleading; it is adaptive *closure* of backlog. A future gateway that wants to coordinate rather than compete with vLLM may need multiple concurrent backend calls, token-level feedback, or an engine-native scheduling interface. Those are different designs from a black-box HTTP batcher.

6.2 Practical decision rule

For a gateway operator, the first question should be whether direct vLLM already achieves the offered load within the latency objective. If it does, outer batching must demonstrate an end-to-end improvement, not merely a larger batch size. At low λw , a fixed window has little opportunity to find companions and should default to pass-through. At high λS , a serial gateway will coalesce backlog automatically, but operators should inspect queue growth and tail latency before treating this as a throughput optimization. The safest default is therefore engine-owned batching with gateway instrumentation; enable outer coalescing only when paired measurements show that reduced backend cost exceeds added queueing for the target workload.

6.3 Threats to validity

Hardware, runtime, and order. The study uses one consumer GPU under WSL2 and the Windows display-driver model. Absolute latency and host scheduling may differ on native Linux or datacenter GPUs. Conditions were executed in a fixed, recoverable order rather than randomized or interleaved, and collection spanned multiple sessions after documented interruptions. Warmup and cooldown do not remove slow temporal or thermal confounding. Paired seeds reproduce workload construction but do not pair wall-clock state. This weakens small cross-mode comparisons; it is less threatening to the within-run identity between queue growth, backend occupancy, and outer grouping. GPU monitoring also lost `nvidia-smi` access after a restart, so the available samples cannot support a complete utilization, VRAM, power, or thermal comparison.

Model and workload. `Qwen2.5-1.5B-Instruct` is small enough to run reproducibly on 12 GB VRAM. Larger models, long contexts, quantization, tensor parallelism, and speculative decoding can change service time and the balance between prefill and decode. The primary workload uses short, synthetic prompts, constant arrivals, and a 64-token output cap. Real traces are bursty and heterogeneous [6]; our result should not be generalized to them without the planned sensitivity study.

Load range. The evaluated rates are calibration-derived points up to 1.8 requests/s. They test a regime in which direct service accepts all offered traffic, not maximum sustainable saturation. This is deliberate for testing the hypothesis, but it does not establish the gateway’s overload capacity.

Statistics. Three repetitions support paired mechanism evidence but only coarse uncertainty estimates. Student- t intervals with two degrees of freedom are sensitive to any outlying run. We therefore show individual run points and avoid significance language unsupported by the interval.

Implementation boundary. ADIP uses one coordinator that awaits a backend batch. The measured backlog mechanism is specific to this transparent, serial outer architecture. A pipelined or concurrent gateway could expose arrivals to vLLM sooner, while an engine-integrated policy could access token-level state. The paper characterizes a common and intentionally simple design, not every possible gateway.

Metric boundary. End-to-end request latency is the principal outcome. The completions API does not expose token streaming in this experiment, so we do not separately report time to first token and time per output token. A system optimized for interactive streaming may require those objectives. The reported requests/s metric is successful request count divided by the fixed arrival-schedule duration; it verifies completion of offered work but is not a service-capacity measurement because it ignores drain time after the final scheduled arrival.

6.4 Reproducibility as a result

The artifact separates exploratory pilot evidence from final claims, freezes policy parameters before the primary matrix, records every exclusion, and computes uncertainty across runs. This discipline matters for small-system research: a negative result with a complete evidentiary trail is more informative than a large percentage derived from one favorable run. The code, configs, tests, raw-schema definitions, final aggregate JSON, and analysis scripts make every number in the paper traceable to named run IDs. Raw traces remain local until an archival release is prepared; the manuscript does not claim that a public dataset already exists.

7 Conclusion

Gateway micro-batching in front of a continuously batching LLM engine is not a free second optimization. It is a second scheduler with less information and its own queue. This paper separates two mechanisms that are easy to conflate: companions found during an intentional micro-window and backlog accumulated while a serial outer worker awaits the backend. ADIP turns that distinction into a falsifiable, request-level experiment on a single consumer-grade GPU.

Across the 48-condition experiment, fixed batching forms larger calls at 1.0–1.8 requests/s but adds 1.05–1.51 seconds of paired p95 latency relative to pass-through. A measured-occupancy model predicts those groups with 0.86% mean absolute percentage error, while queueing adds 0.77–0.83 seconds and backend time does not fall consistently. Pass-through overhead and the adaptive policy’s advantage remain unresolved under the run-level intervals.

Within this tested scope, the practical result is precise: a larger outer batch is not evidence of useful batching. When it is created by serial gateway occupancy, it can be a measurement of withheld scheduling opportunity. The artifact makes that result auditable and provides a predeclared base for randomized, bursty, heterogeneous, and larger-model validation.

References

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in llm inference with sarathi-serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134. USENIX Association, 2024. URL <https://www.usenix.org/conference/osdi24/presentation/agrawal>.
- [2] Daniel Crankshaw, Xin Wang, Giulio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. Clipper: A low-latency online prediction serving system. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 613–627. USENIX Association, 2017. URL <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/crankshaw>.
- [3] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, and Yuxiong He. Deepspeed-fastgen: High-throughput text generation for llms via mii and deepspeed-inference. *arXiv preprint arXiv:2401.08671*, 2024. URL <https://arxiv.org/abs/2401.08671>.
- [4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. URL <https://arxiv.org/abs/2309.06180>.
- [5] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025. URL <https://arxiv.org/abs/2412.15115>.

- [6] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. Burstgpt: A real-world workload dataset to optimize llm serving systems. *arXiv preprint arXiv:2401.17644*, 2024. URL <https://arxiv.org/abs/2401.17644>.
- [7] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538. USENIX Association, 2022. URL <https://www.usenix.org/conference/osdi22/presentation/yu>.
- [8] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210. USENIX Association, 2024. URL <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>.

A Reproduction checklist

The artifact pins the model revision, Python and vLLM environments, generation parameters, workload seeds, and experiment configuration. Each run directory contains a manifest, append-only request records, GPU samples, and a generated summary. The analysis validates all declared conditions before producing figures or tables. Raw invalid or interrupted attempts remain in dated archive directories and are never pooled with final runs. The repository includes the exact LuaLaTeX/BibTeX build sequence and a progress ledger that records each execution and recovery decision.